COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212-Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	9/29/26	**DATE SUBMITTED:**	10/1/26
NAME:	Francis Nikko I. Andoy	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1 Install, Configure and Manage Enterprise Log Monitoring via Ansible

1. PROCEDURE OUTPUT

1. Create a playbook that:
a. Install and configure Elastic Stack in separate hosts (Elastic Search, Kibana, Logstash)

Before creating the playbook, we must first create the inventory where in the remote hosts will live: When creating an inventory file the first step is to define the all which is the top level that contains all of the hosts. Followed by the children which consist of the servers including::

```
all:
  children:
    elastic_servers:
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.102
          ansible_user: server-1

    kibana_servers:
      hosts:
        ubuntu2:
          ansible_host: 192.168.56.103
          ansible_user: server-2

    logstash_servers:
      hosts:
        centos1:
          ansible_host: 192.168.56.106
          ansible_user: cent-server
```

To check whether the inventory is grouped successfully we can use –list or --graph

```
(base) nikko@Nikko:~/activity_repo/activity_10$ ansible-inventory -i inventory.yaml --list
{
    "_meta": {
        "hostvars": {
            "centos1": {
                "ansible_host": "192.168.56.106",
                "ansible_user": "cent-server"
            },
            "ubuntu1": {
                "ansible_host": "192.168.56.102",
                "ansible_user": "server-1"
            },
            "ubuntu2": {
                "ansible_host": "192.168.56.103",
                "ansible_user": "server-2"
            }
        },
        "profile": "inventory_legacy"
    },
    "all": {
        "children": [
            "ungrouped",
            "elastic_servers",
            "kibana_servers",
            "logstash_servers"
        ]
    },
    "elastic_servers": {
        "hosts": [
```

The output should be something like this, the remote servers are organized.

We can also check if the ssh connectivity is working by issuing the command:
Ansible all -m ping

```
(base) nikko@Nikko:~/activity_repo/activity_10$ ansible all -m ping
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of
 another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-c
ore/2.19/reference_appendices/interpreter_discovery.html for more information.
centos1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ubuntu2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
```

The output should be like where all of the remote should have a green text indicating that the connectivity is working.

Next would be to create the ansible.cfg so that we don't have to explicitly define the inventory.

```
[defaults]

inventory = ./inventory.yaml
host_key_checking = False
```

The output should be like this:

```
(base) nikko@Nikko:~/activity_repo/activity_10$ ansible-inventory --list
{
    "_meta": {
        "hostvars": {
            "centos1": {
                "ansible_host": "192.168.56.106",
                "ansible_user": "cent-server"
            },
            "ubuntu1": {
                "ansible_host": "192.168.56.102",
                "ansible_user": "server-1"
            },
            "ubuntu2": {
                "ansible_host": "192.168.56.103",
                "ansible_user": "server-2"
```

In the given screen shot we didn't have to explicitly define the inventory.yaml since we created the ansible.cfg

The next step would be to create the site.yml which is for orchestrating the whole roles system later.

```
---
- hosts:  all
  become: true
  roles:
    - role: path

- hosts: elastic_servers
  become: true
  roles:
    - role: elastic
      tags: elastic

- hosts: kibana_servers
  become: true
  roles:
    - role: kibana
      tags: kibana

- hosts: logstash_servers
  become: true
```

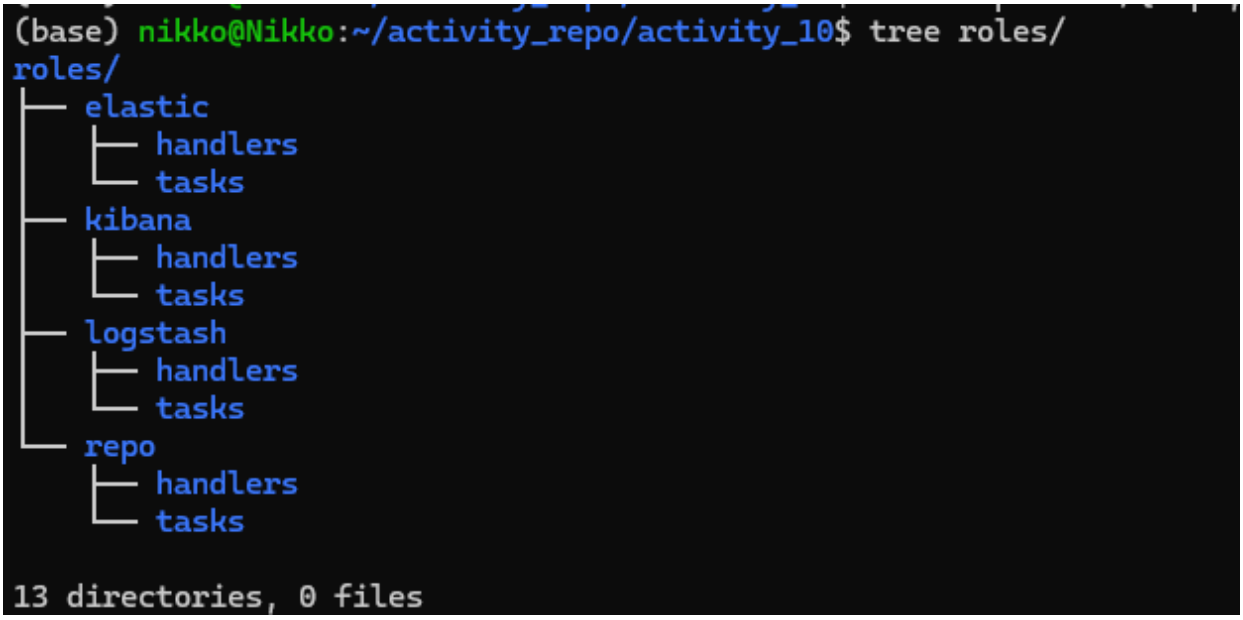
```
roles:
  - role: logstash
    tags: logstash
```

To check whether the syntax is correct we can use the command ansible-playbook --syntax-check site.yml

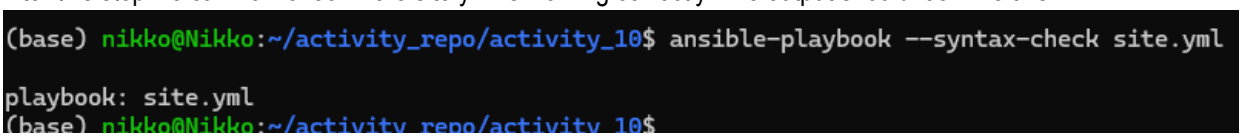
Note:
Before checking if the site.yml's syntax is correct make sure to create the folder structure that contains the tasks and handlers. In the site.yml the role: path should have the tasks and handlers directory each.

To create the directory issue the command:
mkdir -p roles/{repo,elastic,kibana,logstash}/{tasks,handlers}

To check the folder structure easily just download the tree using sudo apt install tree:
Then issue the command tree roles the output should be like this:



After this step we can now check if the site.yml is working correctly. The output should look like this:



Now that the site is working correctly the next step would be to create the main.yml in each sub directory of the roles/

First is the repo which contains the key, package repo, and Installs Baseline System Utilities & Updates Caches

```yaml
---
#UBUNTU
- name: Install APT prerequisites
  ansible.builtin.apt:
    name: [apt-transport-https, ca-certificates, gnupg]
    state: present
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- name: Add Elastic GPG key
  ansible.builtin.apt_key:
    url: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    state: present
  when: ansible_distribution == "Ubuntu"

- name: Add Elastic APT repository
  ansible.builtin.apt_repository:
    repo: "deb https://artifacts.elastic.co/packages/8.x/apt stable main"
    state: present
    filename: elastic-8.x
  when: ansible_distribution == "Ubuntu"

#CENTOS

- name: Import Elastic GPG Key
  ansible.builtin.rpm_key:
    key: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    state: present
  when: ansible_distribution == "CentOS"

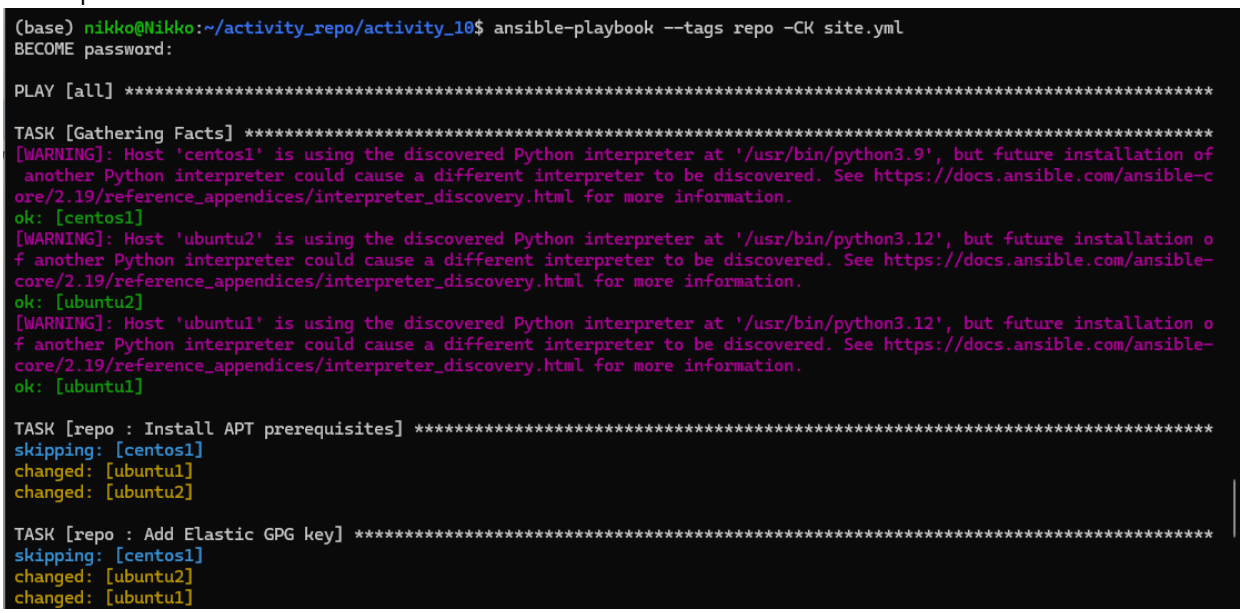
- name: Add Elastic YUM repository
  ansible.builtin.yum_repository:
    name: elastic-8.x
    description: "Elasticsearch repository for 8.x packages"
    baseurl: "https://artifacts.elastic.co/packages/8.x/yum"
    gpgcheck: yes
    gpgkey: https://artifacts.elastic.co/GPG-KEY-elasticsearch
    enabled: yes
```

```
  when: ansible_distribution == "CentOS"
```

To see whether the role repo is working correctly we can issue the command:
Ansible-playbook --tags <name_of_tag> -CK site.yml

We have to add tags to so that we can execute it independently to the other roles. -CK, C is simply executing the site.yml without changing anything in the remote server. As for the -K since we need to use sudo we have to asked the sudo.

The output should be like this:



Next would be to create the main.yml in both tasks and handlers in elastic.

roles/tasks/Main.yml:

```
---
- name: Configure JVM heap size
  ansible.builtin.copy:
    dest: /etc/elasticsearch/jvm.options.d/heap.options
    content: |
      -Xms512m
      -Xmx512m
    owner: root
    group: elasticsearch
    mode: '0644'

- name: Deploy clean elasticsearch configuration
```

```
  ansible.builtin.copy:
    dest: /etc/elasticsearch/elasticsearch.yml
    content: |
      cluster.name: my-application
      node.name: Server-1
      path.data: /var/lib/elasticsearch
      path.logs: /var/log/elasticsearch
      network.host: 0.0.0.0
      http.port: 9200
      discovery.type: single-node
      xpack.security.enabled: false
      xpack.security.enrollment.enabled: false
      xpack.security.http.ssl.enabled: false
      xpack.security.transport.ssl.enabled: false
    owner: root
    group: elasticsearch
    mode: '0660'
    force: yes

- name: Ensure correct directory ownership
  ansible.builtin.file:
    path: "{{ item }}"
    state: directory
    owner: elasticsearch
    group: elasticsearch
    recurse: yes
  loop:
    - /var/lib/elasticsearch
    - /var/log/elasticsearch

- name: Reload systemd daemon
  ansible.builtin.systemd:
    daemon_reload: yes

- name: Ensure Elasticsearch service is running and enabled
  ansible.builtin.service:
    name: elasticsearch
    state: restarted
```

```
    enabled: true
```

tasks role it to install and configure the necessary files for elastic.

As for the handlers the role is to restart the service since not all services are started as their defaut.

roles/handlers/main.yml

```
---
- name: restart_elasticsearch
  ansible.builtin.service:
    name: elasticsearch
    state: restarted
```

Now we just need to check if both files are working by issuing the command: ansible-playbook --tags <name_of_tags> -CK site.yml

For sure there will be a problem since before running the elastic the package manager should also be updated and the key should also be there. In conclusion we can't check if the playbook is correct by using -C since there's some pre req that we need to have before finishing the actual playbook.

To fix the problem we just need to execute the playbook. By issuing the command:
Ansible-playbook --tags <name_of_tags> -K site.yml

Output of each roles

Repo:

```
(base) nikko@Nikko:~/activity_repo/activity_10$ ansible-playbook --tags repo -K site.yml
BECOME password:

PLAY [all] ********************************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
f[WARNING]: Host 'ubuntu2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation
of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible
-core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu2]
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation o
f another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-
core/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu1]
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of
 another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-c
ore/2.19/reference_appendices/interpreter_discovery.html for more information.
ok: [centos1]

TASK [repo : Install APT prerequisites] ***************************************************************************
skipping: [centos1]
changed: [ubuntu2]
changed: [ubuntu1]
```

Elastic role:

```
TASK [elastic : Deploy clean elasticsearch configuration] *********************************************************
changed: [ubuntu1]

TASK [elastic : Ensure correct directory ownership] ***************************************************************
ok: [ubuntu1] => (item=/var/lib/elasticsearch)
ok: [ubuntu1] => (item=/var/log/elasticsearch)

TASK [elastic : Reload systemd daemon] ****************************************************************************
ok: [ubuntu1]

TASK [elastic : Ensure Elasticsearch service is running and enabled] **********************************************
changed: [ubuntu1]

PLAY [kibana_servers] *********************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [ubuntu2]

PLAY [logstash_servers] *******************************************************************************************

TASK [Gathering Facts] ********************************************************************************************
ok: [centos1]

PLAY RECAP ********************************************************************************************************
centos1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu1                    : ok=7    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

(base) nikko@Nikko:~/activity_repo/activity_10$
```

Check if the elasticsearch is functioning correctly:

```
(base) nikko@Nikko:~/activity_repo/activity_10$ curl -X GET http://192.168.56.102:9200
{
  "name" : "Server-1",
  "cluster_name" : "my-application",
  "cluster_uuid" : "B3mmAUQISdGIeLIP2xRJcQ",
  "version" : {
    "number" : "8.19.22",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "3b2a41103de35e0af4064d647974032fcc1bcde9",
    "build_date" : "2026-09-18T10:10:07.018982263Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
(base) nikko@Nikko:~/activity_repo/activity_10$
```

Issue the command curl -X GET http://192.168.56.102

If there's a error regarding (28) Failed to connect to 192.168.56.102 port 9200 after 134999 ms: Could not connect to server

We must first add the port 9200 to the firewall by issuing the command
Ufw allow 9200/tcp

Next is to create the logstash role:

```
TASK [Gathering Facts] *********************************************************************************************
ok: [centos1]

TASK [logstash : Install Logstash and OpenJDK] *********************************************************************
changed: [centos1]

TASK [logstash : Create basic beats-to-elasticsearch logstash pipeline] ********************************************
changed: [centos1]

TASK [logstash : Ensure Logstash is started and enabled] ***********************************************************
changed: [centos1]

RUNNING HANDLER [logstash : restart_logstash] **********************************************************************
changed: [centos1]

PLAY RECAP *********************************************************************************************************
centos1                    : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

To check weather the logstash is working properly we can issue the command ansible logstash_servers -m command -a "curl -X GET http://localhost:9600/?pretty" -b -K

The output should look like this:

```
(base) nikko@Nikko:~/activity_repo/activity_10$ ansible logstash_servers -m command -a "curl -X GET http://localhost:960
0/?pretty" -b -K
BECOME password:
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of
 another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-c
ore/2.19/reference_appendices/interpreter_discovery.html for more information.
centos1 | CHANGED | rc=0 >>
{
  "host" : "localhost.localdomain",
  "version" : "8.19.22",
  "http_address" : "127.0.0.1:9600",
  "id" : "2c54ac04-033f-4eec-903f-bbcdd27c9161",
  "name" : "localhost.localdomain",
  "ephemeral_id" : "0454a734-4036-4541-9632-47b462c9cca5",
  "snapshot" : false,
  "status" : "green",
  "pipeline" : {
    "workers" : 2,
    "batch_size" : 125,
    "batch_delay" : 50
  },
  "build_date" : "2026-09-18T17:14:57+00:00",
  "build_sha" : "dbf5917ccbe232a50f8b10e28552bf939f2d2f40",
  "build_snapshot" : false
}  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   510  100   510    0     0    228      0  0:00:02  0:00:02 --:--:--   229
(base) nikko@Nikko:~/activity_repo/activity_10$
```

The last step would be to create the script for kibana

```
TASK [kibana : Install Kibana] ************************************************************************************
ok: [ubuntu2]

TASK [kibana : Allow external traffic on port 5601] ***************************************************************
ok: [ubuntu2]

TASK [kibana : Point Kibana to Elasticsearch host] ****************************************************************
changed: [ubuntu2]

TASK [kibana : Ensure Kibana is started and enabled] **************************************************************
changed: [ubuntu2]

RUNNING HANDLER [kibana : restart_kibana] *************************************************************************
changed: [ubuntu2]

PLAY [logstash_servers] *******************************************************************************************

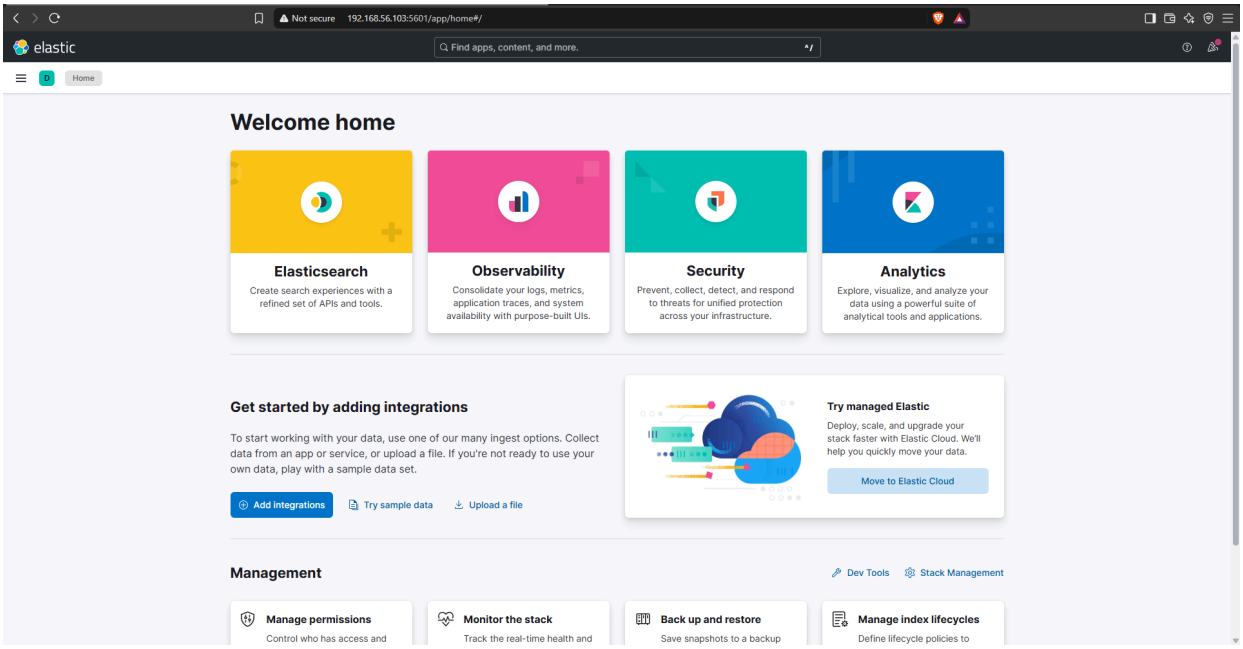
TASK [Gathering Facts] ********************************************************************************************
ok: [centos1]

PLAY RECAP ********************************************************************************************************
centos1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu2                    : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

(base) nikko@Nikko:~/activity_repo/activity_10$
```

Before running the kabina make sure that the firewall in that specific server where kabina is running is allowed if not even if the script is running the http will return unreachable.

Confirmation:



This is running in my server 2 which is just ubuntu since I changed my approach into distributing the load in my other servers. My centos is for logstash, ubuntu1 for elastic and ubuntu2 is for kibana.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. What are the benefits of having a log monitoring tool?
   - Having a log monitor tool will make the work a lot more easier since we don't have to manually oversee the changes in our services. Where any seconds anything can change we will just be notified if the service is down, we can as soon as the notification is sent to us.

conclusion:
In conclusion, this activity helped me understand how ansible works from scratch. Using commands to see if the script was implemented correctly and the connections of nodes to the manager node. I also learned how the roles works to simplify and to organize the automation. For example, to use tasks so that we don't have to define it inside the site.yml we just need to create the folder for it. Since I'm having a problem with downloading both elastic on both servers I solved this problem by using my centos for logstash. I distributed the installations that I need to do to my 3 servers to lessen the burden on my servers.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts, summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Give my own understanding to ai to if I understand the code and help me debug the problem.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.